Centralized Real-Time Queue Management Platform

Tech Stack Specification — `techstack.md`

---

1. Document Information

Field	Details
Project	Centralized Real-Time Queue Management Platform
Document	techstack.md
Version	1.0
Status	Client-Selected Stack
Frontend	Next.js

| Backend | Node.js |

| Database | PostgreSQL |

| Architecture | Full-Stack Web Application |

| Application Type | Multi-Tenant Queue Management Platform |

---

2. Technology Stack Summary

The project will use the following core technologies selected by the client:

```text
Frontend
  ↓
```

Next.js

Backend

↓

Node.js

Database

↓

PostgreSQL

The application will be designed around these technologies while keeping the system secure, maintainable, modular, and scalable.

Additional supporting technologies will be selected only when required by the project requirements.

---

3. Frontend — Next.js

3.1 Purpose

Next.js will be used to build the web application and all responsive user

interfaces.

The frontend will provide interfaces for:

- End Users
- Queue Operators
- Organization Administrators
- Platform Administrators, where required

3.2 Responsibilities

Next.js will be responsible for:

- Application pages and routing
- UI rendering
- Responsive layouts
- Forms and user interactions
- Client-side validation
- Queue status display
- Real-time queue status updates
- Operator dashboard
- Admin dashboard
- Authentication/session UI
- Loading states
- Error states
- User feedback and notifications
- Responsive mobile and desktop interfaces

The frontend must not be responsible for authoritative queue calculations or security decisions.

---

4. Responsive Design

Responsive design is a mandatory requirement.

The application must work correctly across:

Mobile

↓

Tablet

↓

Laptop

↓

Desktop

4.1 End-User Interface

The end-user experience should prioritize:

Mobile → Tablet → Desktop

Users will commonly access the application through their mobile phones after scanning a QR code.

The following must work properly on mobile:

- QR-based queue joining
- Registration form
- Queue position
- Estimated waiting time
- Queue status
- Leave/cancel queue
- Notifications/status messages

4.2 Operator/Admin Interface

Operator and administrator interfaces should prioritize:

> Laptop → Desktop → Tablet → Mobile

The dashboards must remain usable on smaller screens as well.

---

5. Backend — Node.js

5.1 Purpose

Node.js will be used as the backend application layer.

The backend will contain the application's authoritative business logic and APIs.

5.2 Responsibilities

Node.js will be responsible for:

- Queue management
- Queue number assignment
- Queue position management
- Queue state transitions
- Marking users as DONE
- Removing completed users from the active queue
- Admin queue reordering
- Estimated waiting-time calculation
- User registration
- Authentication
- Authorization
- Role-based access control
- Multi-tenant access control
- Real-time update handling
- WhatsApp/SMS notification triggering
- API validation
- Rate limiting
- Error handling
- Logging
- Audit operations

---

6. Backend as the Source of Truth

The backend must be the authoritative source for all important queue operations.

The frontend must never be trusted to determine or enforce queue state.

Frontend

↓

Node.js API

↓

Validate

↓

Authorize

↓

Business Logic

↓

PostgreSQL

The backend determines:

- Current queue position
- Queue number
- Whether a user can join
- Whether a user can leave
- Whether an operator can mark an entry DONE

- Whether an administrator can modify queue order
- Whether a queue is active
- Whether a request belongs to the correct tenant
- Whether an operation is authorized

---

7. Database — PostgreSQL

PostgreSQL will be the primary relational database.

It will provide persistent storage and transactional consistency for the application.

Potential core entities include:

- Organizations / Tenants
- Users
- Roles
- Queues
- Queue Entries
- Queue Events
- Notifications
- Notification Delivery Records
- Configuration
- Audit Events

Detailed database design will be documented separately in:

database.md

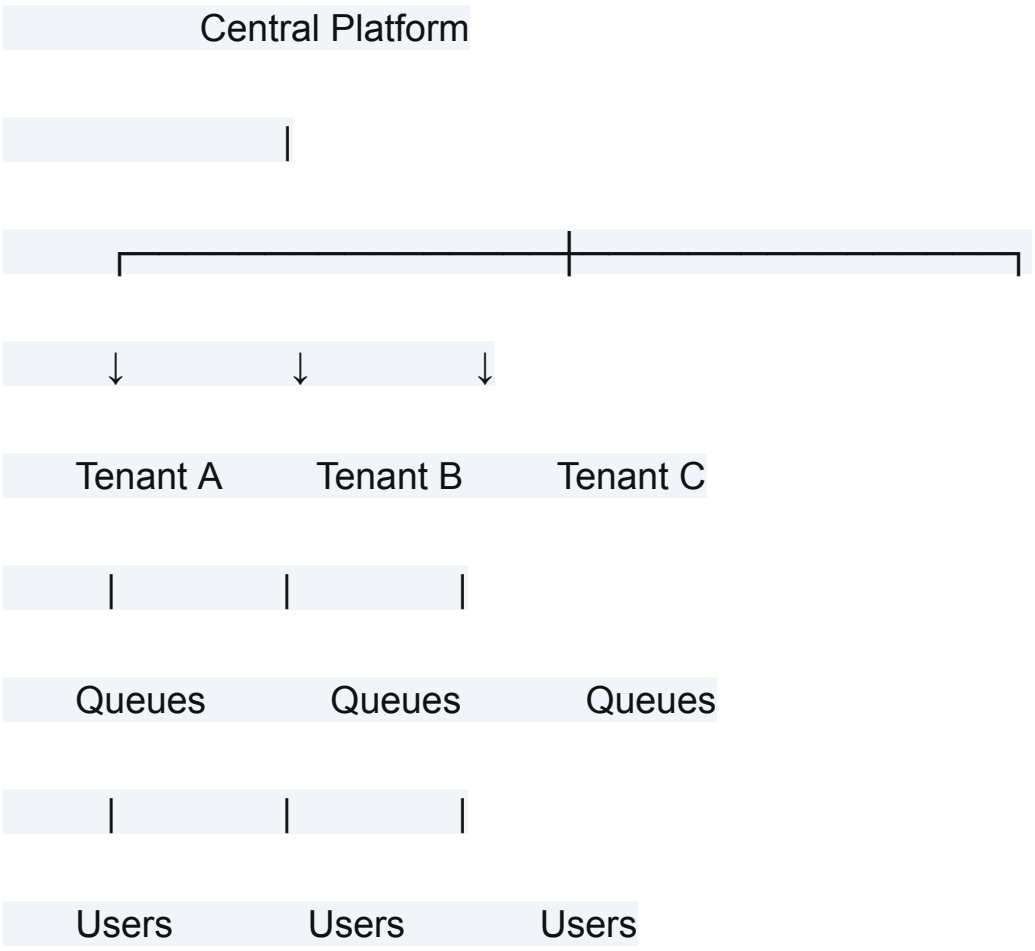
---

8. Multi-Tenant Architecture

Multi-tenancy is a mandatory requirement.

The platform must support multiple organizations using the same application infrastructure while keeping their data strictly isolated.

Conceptually:



Each tenant must operate independently.

---

9. Tenant Data Isolation

A tenant must never be able to access another tenant's data.

For example:

Tenant A

↓

Tenant A Data ✓ Allowed

Tenant A

↓

Tenant B Data ✗ Blocked

Tenant isolation must be enforced by the backend and database-access layer.

It must not depend only on frontend filtering or routing.

The final database isolation strategy will be documented in `database.md`.

---

10. Multi-Tenant Database Strategy

Proposed Approach

The initial approach is:

Shared Database

+

Shared Schema

+

Tenant Identification

Tenant-owned records will contain appropriate tenant identification where required.

This approach is **proposed and subject to final confirmation in `database.md`**.

The final decision will consider:

- Security
- Data isolation
- Performance
- Scalability
- Query complexity
- Database maintenance
- Backup and recovery requirements

---

11. Queue Management

The default queue behavior will be:

First Come

↓

First Served

Example:

Person A → #1

Person B → #2

Person C → #3

Person D → #4

When a new user joins, the system automatically assigns the next queue position.

---

12. Queue Completion

When an operator clicks:

DONE

the backend will:

1. Verify the operator's identity.
2. Verify tenant access.
3. Verify queue permissions.
4. Verify the queue entry's current state.
5. Mark the queue entry as completed.
6. Remove it from the active queue.
7. Update the remaining active queue positions.
8. Record the queue event.
9. Publish the updated queue state in real time.
10. Trigger applicable notifications.

Example:

Before:

#1 Person A

#2 Person B

#3 Person C

#4 Person D

Person A → DONE

After:

#1 Person B

#2 Person C

#3 Person D

---

13. Admin Queue Reordering

The normal queue is First-Come-First-Served.

However, an authorized Organization Admin can manually change a user's position when required.

This allows VIP/priority handling without creating a separate queue system.

Example:

Before:

#1 A

#2 B

#3 C

#4 D

Admin changes D → #2

After:

#1 A

#2 D

#3 B

#4 C

The backend must perform reordering safely and consistently.

All affected users must receive the updated queue state in real time.

---

14. Estimated Waiting Time

The application will provide an approximate waiting-time estimate.

The estimate will be based on the actual time taken by previously completed queue entries.

Conceptually:

Previous Completed Entries

↓

Actual Service Duration

↓

Average Service Time

↓

People Ahead of Current User

↓

Approximate Waiting Time

Example:

Previous service times:

8 minutes

10 minutes

7 minutes

9 minutes

Average ≈ 8.5 minutes

Current user:

Position #4

People ahead = 3

Estimated wait ≈

3 × 8.5

≈ 25.5 minutes

The estimate is approximate and may change as queue activity changes.

The exact calculation, rolling-average strategy, and handling of insufficient historical data will be finalized in `database.md` and `architecture.md`.

---

15. Real-Time Queue Updates

Real-time updates are a core requirement.

When the queue changes, connected users should receive the latest queue state without manually refreshing the page.

Example:

Operator clicks DONE

↓

Node.js Backend

↓

Database Updated

↓

Real-Time Update

↓

Connected Users

↓

Queue Position Updated

Example:

#4

↓

#3

↓

#2

↓

#1

The exact real-time implementation will be finalized in `architecture.md`.

A WebSocket-based approach is a strong candidate, but the specific library/implementation will not be locked until architecture evaluation.

---

16. Real-Time Connection Recovery

If a user's connection is temporarily lost:

Connection Lost

↓

Automatic Reconnection

↓

Fetch Latest Queue State

↓

Resume Real-Time Updates

The database/backend remains the source of truth.

A temporary network problem must not cause queue state loss.

---

17. Notification Architecture

The platform must support both:

- WhatsApp
- SMS

Notifications should be implemented through a separate notification

abstraction/service layer.

```
Queue Event

    ↓

Notification Service

    |

    ├── WhatsApp

    |

    └── SMS
```

This keeps notification logic separate from the core queue logic and allows notification providers to be changed later if required.

---

18. Notification Configuration

WhatsApp and SMS functionality must be configurable through environment/configuration.

For example:

```
WHATSAPP_ENABLED=true

SMS_ENABLED=true
```

Provider-specific configuration will also be stored through environment variables or secure configuration management.

Actual providers will be selected during implementation.

No specific provider is locked at this stage.

---

19. Notification Events

External notifications should be used for important queue events rather than every high-frequency real-time position update.

Potential notification events include:

- Queue joined
- Turn approaching
- Turn available
- Queue cancelled/closed
- Queue suddenly clears
- Other important queue events defined by the final product requirements

Real-time web updates and external notifications are separate mechanisms.

Example:

Web UI:

#4 → #3 → #2 → #1

External notification:

"Your turn is now available."

---

20. QR Code Management

Only an authorized **Organization Admin** can generate a queue QR code.

End users can only scan the QR code.

Flow:

Organization Admin

↓

Creates Queue

↓

Generates QR Code

↓

QR displayed/printed

↓

User scans QR

↓

User joins queue

The QR code must be associated with the correct tenant and queue.

Unauthorized users must not be able to generate or modify queue QR codes.

---

21. User Queue Joining

The standard joining flow is:

User

↓

Scan Admin-generated QR

↓

Queue Registration Page

↓

Enter Required Details

↓

Submit

↓

Backend Validation

↓

Join Queue

↓

Automatically Receive Queue Position

Example:

First user → #1

Second user → #2

Third user → #3

...

Fiftieth user → #50

The queue number must be assigned by the backend.

---

22. User Leaving the Queue

Users can leave/cancel their queue entry.

Before cancellation, the system must clearly warn the user that they will lose their current queue position.

Example:

Are you sure you want to leave the queue?

You will lose your current position.

[Cancel]

[Leave Queue]

After confirmation:

User Removed

↓

Active Queue Updated

↓

Remaining Users Move Forward

---

23. Queue Clearing

If the queue suddenly clears or the user's queue situation changes significantly, the appropriate users must receive a notification.

For a completely cleared queue:

Queue Cleared

↓

Queue State Updated

↓

Affected Users Notified

---

24. Authentication

Privileged functionality must require authentication.

Authentication will apply to appropriate:

- Organization Administrators
- Queue Operators
- Platform Administrators

End users should be able to join and track their queue without requiring a traditional long-term account unless otherwise required by the final product requirements.

---

25. Authorization

Authorization must be enforced by the backend.

Role	Main Access
Platform Admin	Global platform/tenant administration where required
Organization Admin	Tenant management, queue management, QR generation, queue overrides
Queue	Queue operations and marking entries DONE

Operator	
End User	Join queue and view/manage their own queue entry

The frontend must not be treated as an authorization boundary.

---

26. API Security

The backend must protect APIs using:

- Authentication
- Role-based authorization
- Tenant access verification
- Schema validation
- Rate limiting
- Secure error handling
- Logging
- Appropriate audit records

---

27. Rate Limiting

Rate limiting is mandatory.

Limits should be applied according to endpoint sensitivity.

Example:

Queue Join API

→ Strict rate limit

Authentication API

→ Strict rate limit

Admin APIs

→ Restricted rate limit

Public Queue Status

→ Higher appropriate limit

The exact numerical limits will be finalized during security and architecture design.

---

28. Validation

Validation must exist on both frontend and backend.

Frontend

↓

User-friendly validation

↓

Backend

↓

Authoritative schema validation

↓

Business Logic

↓

Database

The backend must never trust frontend input.

Shared validation schemas may be used where appropriate.

---

29. Error Handling

The application must use consistent error handling across frontend and backend.

The system must handle cases such as:

- Invalid input
- Invalid QR code
- Queue unavailable
- Unauthorized action
- Forbidden action
- Queue entry not found
- Rate limit exceeded
- Database errors
- Notification failures
- Real-time connection failures

Internal implementation details and sensitive information must not be exposed to users.

---

30. Code Quality

The project must maintain consistent code-quality standards.

The development workflow should include:

- ESLint
- Prettier
- TypeScript type checking

- Build validation
- Unused variable detection
- Unused import detection
- Automated tests

---

31. Husky

Husky will be used for Git hooks.

Example:

git commit

↓

Husky

↓

Lint

↓

Type Check

↓

Tests

↓

PASS → Commit Allowed

FAIL → Commit Blocked

The exact Git hook configuration will be finalized during project setup.

---

32. Testing Strategy

Testing must include:

- Unit tests
- Integration tests
- End-to-end tests
- API tests
- Multi-tenant isolation tests
- Queue state transition tests
- Real-time behavior tests
- Notification tests
- Security tests
- Responsive UI tests

Every important feature must include:

Happy Flow

+

Negative Flow

---

33. Playwright

Playwright will be the primary browser end-to-end testing framework.

Important flows include:

Admin creates queue

↓

Admin generates QR

↓

User scans QR

↓

User joins queue

↓

Operator performs queue action

↓

Queue position changes

↓

User receives updated state

↓

User's turn becomes available

---

34. Selenium

Selenium will be used where required for additional browser compatibility or specific testing requirements.

Playwright remains the primary E2E testing framework.

The project should avoid unnecessary duplication between Playwright and Selenium.

---

35. UI Component Library — Shadcn UI

Shadcn UI components should be used for the application's reusable UI system.

Components may include:

- Buttons

- Inputs
- Forms
- Dialogs
- Dropdowns
- Tables
- Status indicators
- Toasts
- Alerts
- Navigation components

The component system must support both mobile and desktop interfaces.

---

36. Temporary UI Preview

A temporary UI preview page may be created during development to validate:

- Mobile layout
- Desktop layout
- Queue status
- Queue progression
- Operator dashboard
- Admin dashboard
- Forms

The preview page must be removed before production if it is not part of the actual product.

---

37. Environment Configuration

Environment-specific configuration must be separated from application code.

Configuration may include:

- Database configuration
- Authentication secrets
- WhatsApp configuration
- SMS configuration
- Application URLs
- Rate limits
- Environment settings
- Third-party provider configuration

Secrets must never be hard-coded.

---

38. Secrets Security

The following must never be committed to Git:

- Database passwords
- API keys
- WhatsApp credentials
- SMS credentials
- Authentication secrets
- Production secrets
- `.env` files containing secrets

A proper `.gitignore` must be maintained.

Example:

```
.env

.env.*

!.env.example
```

The exact Git ignore rules will be finalized during project setup.

---

39. Git Branching Strategy

Development should use separate Git branches.

Example:

```
main

|
```

├── feature/queue-management

├── feature/notifications

├── feature/multitenancy

└── feature/admin

The `main` branch should contain stable code.

Meaningful completed features and stable checkpoints should be committed.

---

40. Monorepo Evaluation

Because the project contains a Next.js frontend and Node.js backend, a monorepo is a strong candidate.

The monorepo decision should consider:

- Shared TypeScript types
- Shared validation schemas
- Shared UI components
- Testing
- Build process
- CI/CD
- Deployment
- Developer experience
- Project complexity

The final monorepo tooling will be selected after technical evaluation.

---

41. Proposed Monorepo Structure

A possible structure is:

```
project/
│
├── apps/
│   ├── web/                # Next.js application
│   └── api/                # Node.js backend
│
├── packages/
│   ├── ui/                 # Shared UI components
│   ├── types/              # Shared TypeScript types
│   ├── validation/         # Shared validation schemas
│   └── config/             # Shared configuration
```

│

├── tests/

│

├── prd.md

├── techstack.md

├── database.md

├── architecture.md

└── development.md

This structure is proposed and may be adjusted after architecture review.

---

42. Shared Type Contracts

Shared TypeScript types should be considered for common frontend/backend entities.

Examples:

Organization

User

Queue

QueueEntry

QueueStatus

Notification

This helps maintain consistency between the frontend and backend.

---

43. Shared Validation

Where appropriate, validation schemas can be shared between frontend and backend.

Example:

Registration Schema

|

├── Next.js Frontend

|

└── Node.js Backend

Backend validation remains authoritative.

---

44. Queue Domain Logic

Queue business logic must remain on the backend.

This includes:

- Queue number assignment
- Queue position updates
- DONE processing
- Queue removal
- Admin reordering
- Waiting-time calculation
- Queue state transitions
- Notification triggers

The frontend should display backend-controlled queue state.

---

45. Technology Responsibilities

Technology / Component	Responsibility
Next.js	Web application, UI, routing, responsive interfaces

Node.js	Backend API and business logic
PostgreSQL	Persistent relational data storage
Real-Time Layer	Live queue-state synchronization
WhatsApp Integration	WhatsApp customer notifications
SMS Integration	SMS customer notifications
Husky	Git hook automation
Playwright	Primary browser E2E testing
Selenium	Additional browser testing where required
Shadcn UI	Reusable UI component system

---

46. Technology Decision Principles

All additional technology decisions must follow these principles:

Requirement First

Technology must solve an actual project requirement.

Simplicity

Prefer simple and maintainable solutions over unnecessary complexity.

Security

Security must be considered in every architectural decision.

Multi-Tenant Safety

Tenant isolation must remain a fundamental requirement.

Scalability

The system should be capable of supporting increasing tenants, users, and queue activity.

Maintainability

The codebase must remain understandable and maintainable.

Testability

The architecture must support reliable automated testing.

Vendor Flexibility

Third-party notification providers should be replaceable where practical.

---

47. Core Technology Non-Goals

The core platform does not require:

- Native iOS application
- Native Android application
- Restaurant-specific table management
- Hotel room management
- Doctor/resource scheduling
- Counter/desk resource assignment
- Complex resource management

The platform remains a **generic queue management system** that can be used across different industries.

---

48. Technology Stack Acceptance Criteria

The technology stack is considered ready when:

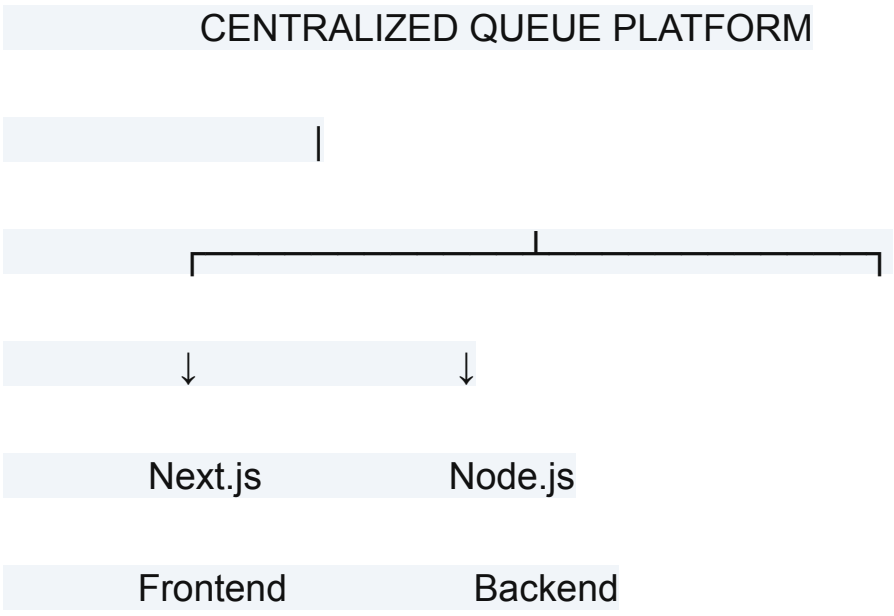
- Next.js confirmed for frontend
- Node.js confirmed for backend
- PostgreSQL confirmed for database
- Multi-tenant architecture supported
- Tenant isolation strategy defined
- Real-time communication approach selected
- WhatsApp integration approach selected
- SMS integration approach selected
- Environment configuration defined
- Authentication approach defined
- Authorization approach defined

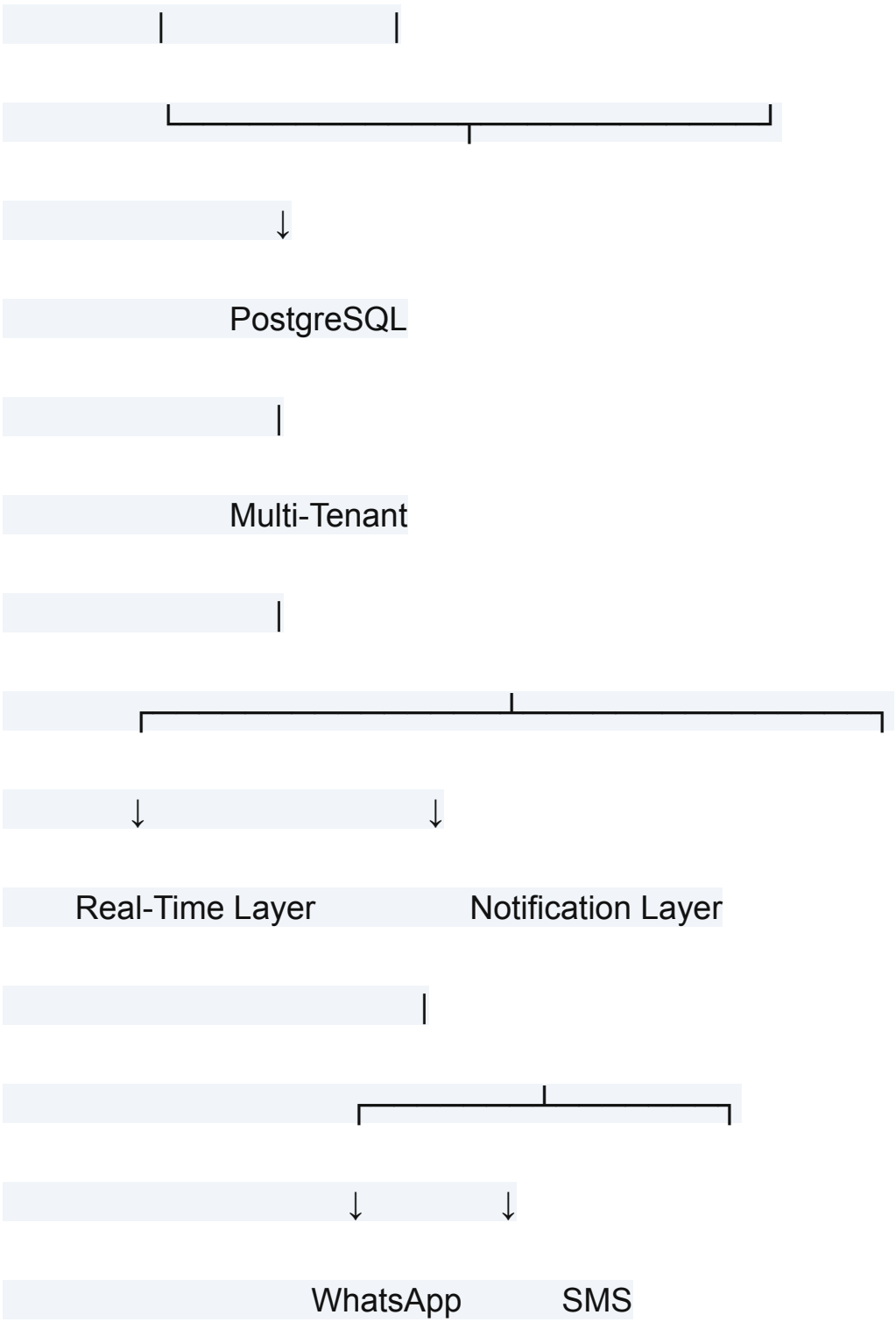
- Rate limiting approach defined
- Validation approach defined
- Testing approach defined
- Playwright scope defined
- Selenium scope defined
- Husky workflow defined
- Shadcn UI usage defined
- Responsive design requirements defined
- Monorepo vs separate repositories evaluated
- Deployment strategy considered
- Logging/monitoring approach considered

---

49. Final Technology Stack

The confirmed client-selected core stack is:




PostgreSQL
Multi-Tenant
Real-Time Layer
Notification Layer
WhatsApp
SMS

Core Stack

Frontend → Next.js

Backend → Node.js

Database → PostgreSQL

Additional supporting technologies will be selected only when required and will be documented in the relevant architecture and development documents.

END OF TECHSTACK DOCUMENT